

Team B: Guardians

Project Criteria: Process Automation

Project Name: System Tracker Automation

Project Overview

The System Tracker Automation System is a Python-based process automation project that automatically monitors and records the health and performance of a computer system. Instead of manually checking system resources, the application continuously tracks CPU usage, RAM usage, disk usage, running processes, and system startup/shutdown events.

The collected information is stored in log files or a database for future analysis. The system also generates performance reports and provides alerts when resource usage exceeds predefined threshold values. This helps users identify system performance issues early and ensures smooth operation of the computer.

The project demonstrates the concept of process automation by performing monitoring, logging, reporting, and alerting tasks automatically without requiring manual intervention.

Project Objectives

1. Automate System Resource Monitoring

The system automatically collects information about computer resources at regular intervals without user involvement.

2. Track CPU, RAM, and Disk Usage

The application continuously monitors processor utilization, memory consumption, and disk space usage to determine system performance.

3. Monitor Active System Processes

The system records information about currently running applications and background processes.

4. Maintain Logs of System Activities

All monitored information is stored in CSV files or a database so that historical records can be analyzed later.

5. Generate Daily Performance Reports

The application summarizes collected data and creates reports containing performance statistics.

6. Send Alerts for High Resource Usage

If CPU or RAM usage exceeds predefined limits, the system generates warning messages or sends email notifications.

Functional Requirements

1. CPU Monitoring

The system should periodically read the current CPU utilization percentage using the psutil library.

Example:

CPU Usage = 45%

This information should be recorded along with the current date and time.

2. RAM Monitoring

The system should continuously monitor memory usage and determine how much RAM is currently being used.

Example:

RAM Usage = 62%

The information should be stored for reporting purposes.

3. Disk Monitoring

The application should monitor disk usage and available storage space.

Example:

Total Disk Space = 500 GB

Used Space = 350 GB

Available Space = 150 GB

This helps identify storage-related issues.

4. Process Monitoring

The system should collect details of active processes running on the computer.

Examples:

- chrome.exe
- python.exe
- explorer.exe

This allows users to understand which applications are consuming system resources.

5. Startup and Shutdown Tracking

The application should detect when the system starts and shuts down.

Example:

Startup Time: 09:00 AM

Shutdown Time: 06:00 PM

These records help calculate system uptime.

6. Data Storage

All monitored information should be stored permanently in:

- CSV files, or
- SQLite database

The data must not be lost when the application closes.

7. Daily Report Generation

At the end of the day, the application should generate a report containing:

- Average CPU usage
- Average RAM usage
- Average Disk usage
- Maximum CPU usage
- Maximum RAM usage

This report provides a summary of system performance.

8. Threshold Checking

The system should compare CPU and RAM usage against predefined limits.

Example:

CPU Threshold = 80%

RAM Threshold = 80%

Whenever these values are exceeded, the system should generate an alert.

9. Alert Generation

If resource usage becomes too high, the application should notify the user.

Example:

WARNING: CPU Usage has exceeded 80%

For advanced implementation, email alerts can also be sent automatically.

10. Log Maintenance

The system should append new records instead of replacing old data.

This ensures historical records remain available for analysis.

11. Error Handling

The application should handle unexpected errors gracefully.

Example:

If a process cannot be accessed, the system should skip that process and continue monitoring instead of terminating.

This improves reliability.

Acceptance Criteria

The project will be considered successful if:

1. CPU monitoring works correctly.
 2. RAM monitoring works correctly.
 3. Disk monitoring works correctly.
 4. Running processes are tracked accurately.
 5. Startup and shutdown events are recorded.
 6. Data is stored successfully in CSV or SQLite.
 7. Daily reports are generated.
 8. Alerts are generated when thresholds are exceeded.
 9. Monitoring continues reliably without crashes.
-

Technologies Used

Python

Main programming language used to develop the application.

psutil

Used to collect system information such as:

- CPU usage
- RAM usage
- Disk usage
- Running processes
- System boot time

CSV

Used to store monitoring logs in CSV files

datetime

Used to record timestamps and generate reports.

smtplib

Used to send automated email alerts when system resource usage exceeds threshold limits.

Project Workflow

Start Application



Monitor CPU Usage



Monitor RAM Usage



Monitor Disk Usage



Collect Running Process Information



Store Data in CSV/Database



Check Threshold Limits



Generate Alerts if Required



Generate Daily Report



Repeat Monitoring Automatically

Conclusion

The System Tracker Automation project is a beginner-friendly process automation application developed using Python. It automates system monitoring tasks, stores performance data, generates reports, and provides alerts when resource usage becomes excessive. The project demonstrates practical implementation of automation, logging, monitoring, reporting, and alert management concepts while remaining simple enough for students and interns to understand and implement.